

```
In [10]: import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\raman\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
Out[10]: True
```

```
In [12]: text= """Tokenization is the first step in text analytics. The
process of breaking down a text paragraph into smaller
chunkssuch as words or sentences is called Tokenization."""
```

```
In [13]: #Sentence Tokenization
from nltk.tokenize import sent_tokenize
tokenized_text= sent_tokenize(text)
print(tokenized_text)
#Word Tokenization
from nltk.tokenize import word_tokenize
tokenized_word=word_tokenize(text)
print(tokenized_word)
```

```
['"Tokenization is the first step in text analytics.', 'The\nprocess of breaking d
own a text paragraph into smaller\nchunkssuch as words or sentences is called Toke
nization.']
[''', 'Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics',
'.', 'The', 'process', 'of', 'breaking', 'down', 'a', 'text', 'paragraph', 'into',
'smaller', 'chunkssuch', 'as', 'words', 'or', 'sentences', 'is', 'called', 'Tokeni
zation', '.']
```

```
In [18]: import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

text = "How to remove stop words with NLTK library in Python?"

# remove special characters
text = re.sub('[^a-zA-Z]', ' ', text)

# tokenize
tokens = word_tokenize(text.lower())

# remove stopwords
filtered_text = []
for word in tokens:
    if word not in stop_words:
        filtered_text.append(word)

print("Tokenized Sentence:",tokens)
print("Filterd Sentence:",filtered_text)
```

```
Tokenized Sentence: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'libr
ary', 'in', 'python']
Filterd Sentence: ['remove', 'stop', 'words', 'nltk', 'library', 'python']
```

```
In [20]: from nltk.stem import PorterStemmer

e_words = ["wait", "waiting", "waited", "waits"]

ps = PorterStemmer()
```

```
for w in e_words:
    rootWord = ps.stem(w)
    print(rootWord)
```

wait  
wait  
wait  
wait

```
In [21]: import nltk
from nltk.stem import WordNetLemmatizer

# create object
wordnet_lemmatizer = WordNetLemmatizer()

# input text
text = "studies studying cries cry"

# tokenization
tokens = nltk.word_tokenize(text)

# Lemmatization
for w in tokens:
    print("Lemma for {} is {}".format(w, wordnet_lemmatizer.lemmatize(w)))
```

Lemma for studies is study  
Lemma for studying is studying  
Lemma for cries is cry  
Lemma for cry is cry

```
In [22]: import nltk
from nltk.tokenize import word_tokenize

data = "The pink sweater fit her perfectly"

words = word_tokenize(data)

for word in words:
    print(nltk.pos_tag([word]))
```

[('The', 'DT')]  
[('pink', 'NN')]  
[('sweater', 'NN')]  
[('fit', 'NN')]  
[('her', 'PRP\$')]  
[('perfectly', 'RB')]

```
In [23]: import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [24]: documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'
```

```
In [25]: bagOfWordsA = documentA.split(' ')
bagOfWordsB = documentB.split(' ')
```

```
In [26]: uniqueWords = set(bagOfWordsA).union(set(bagOfWordsB))
```

```
In [27]: numOfWordsA = dict.fromkeys(uniqueWords, 0)

for word in bagOfWordsA:
    numOfWordsA[word] += 1
```

```
numOfWordsB = dict.fromkeys(uniqueWords, 0)

for word in bagOfWordsB:
    numOfWordsB[word] += 1
```

```
In [28]: def computeTF(wordDict, bagOfWords):
    tfDict = {}
    bagOfWordsCount = len(bagOfWords)

    for word, count in wordDict.items():
        tfDict[word] = count / float(bagOfWordsCount)

    return tfDict

tfA = computeTF(numOfWordsA, bagOfWordsA)
tfB = computeTF(numOfWordsB, bagOfWordsB)
```

```
In [29]: import math
import pandas as pd

# IDF calculation
def computeIDF(documents):
    N = len(documents)
    idfDict = dict.fromkeys(documents[0].keys(), 0)

    for document in documents:
        for word, val in document.items():
            if val > 0:
                idfDict[word] += 1

    for word, val in idfDict.items():
        idfDict[word] = math.log(N / float(val))

    return idfDict

idfs = computeIDF([numOfWordsA, numOfWordsB])

# TF-IDF calculation
def computeTFIDF(tfBagOfWords, idfs):
    tfidf = {}

    for word, val in tfBagOfWords.items():
        tfidf[word] = val * idfs[word]

    return tfidf

tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)

# Convert to DataFrame
df = pd.DataFrame([tfidfA, tfidfB])

print(df)
```

|   | the | is  | Planet   | planet   | fourth   | Mars     | from     | Sun      | \ |
|---|-----|-----|----------|----------|----------|----------|----------|----------|---|
| 0 | 0.0 | 0.0 | 0.138629 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 |   |
| 1 | 0.0 | 0.0 | 0.000000 | 0.086643 | 0.086643 | 0.086643 | 0.086643 | 0.086643 |   |
|   |     |     | Jupiter  | largest  |          |          |          |          |   |
| 0 |     |     | 0.138629 | 0.138629 |          |          |          |          |   |
| 1 |     |     | 0.000000 | 0.000000 |          |          |          |          |   |

In [ ]: